==========

CAPSTONE PROJECT

NOTEBOOK 47: FINAL AUDIO INFERENCE PIPELINE

==========

Purpose:

This notebook is the final deployment-style inference notebook

for the capstone project.

It allows the user to:

1. Load a single audio file

2. Run the frozen final Stage-1 hybrid benchmark

3. Run the frozen final Stage-2 rare-tail fallback layer

4. View candidate-label predictions and rare-tail suggestions

5. Optionally compare with ground truth for FMA tracks

6. Save final report-ready outputs

==========

In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import json
import warnings
from pathlib import Path

import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

```
Seed set to: 42
TensorFlow version: 2.20.0
```

In [2]:

```
# ============================================================
# 2. USER SETTINGS
# ============================================================
# MODE OPTIONS:
# - "existing_fma_track" : use a track_id from your processed FMA tables
# - "external_file"      : use your own audio file

MODE = "external_file"
```

```
#MODE = "existing_fma_track"

# Option 1: choose an FMA track_id that exists in multilabel_full_master_table.csv
#TRACK_ID_TO_TEST = 568

# Option 2: path to your own external file
EXTERNAL_AUDIO_PATH = r"C:\Users\jdevo\Downloads\Bob Marley - Is This Love
(Official Music Video).mp3"

# How many top candidate labels to show
TOP_N_CANDIDATES = 10

# Audio settings
SR = 22050
DURATION = 15
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((DURATION * SR) / HOP_LENGTH)) + 1

print("MODE:", MODE)
print("TOP_N_CANDIDATES:", TOP_N_CANDIDATES)
print("SR:", SR)
print("DURATION:", DURATION)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)
```

```
MODE: external_file
TOP_N_CANDIDATES: 10
SR: 22050
DURATION: 15
N_MELS: 64
MAX_FRAMES: 324
```

In [3]:

```
# ============================================================
# 3. LOAD FROZEN ARTIFACTS
# ============================================================

PROCESSED_DIR = "../data/processed"
MODELS_DIR = "../models"
RAW_METADATA_DIR = "../data/raw/metadata"

# Structured branch
structured_model = joblib.load(f"
{MODELS_DIR}/final_structured_multilabel_candidate150_best_model.joblib")
structured_scaler = joblib.load(f"
{MODELS_DIR}/final_structured_multilabel_candidate150_scaler.joblib")

# Audio branch
audio_model = tf.keras.models.load_model(f"
{MODELS_DIR}/audio_multilabel_candidate150_expanded_final.keras")

# Candidate label columns
candidate_label_cols = np.load(
```

```
    f"{PROCESSED_DIR}/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

# Final frozen config from Notebook 46 / project freeze
with open(f"{PROCESSED_DIR}/final_project_frozen_config.json", "r") as f:
    final_project_config = json.load(f)

STAGE1_STRUCTURED_WEIGHT = float(final_project_config["stage1_structured_weight"])
STAGE1_AUDIO_WEIGHT = float(final_project_config["stage1_audio_weight"])
STAGE1_THRESHOLD = float(final_project_config["stage1_threshold"])

STAGE2_FINAL_STRATEGY = final_project_config["stage2_final_strategy"]
STAGE2_ANCHOR_TRIGGER_THRESHOLD =
float(final_project_config["stage2_anchor_trigger_threshold"])
STAGE2_TOP_K = int(final_project_config["stage2_top_k"])

# Tables
rare_tail_router_df = pd.read_csv(f"
{PROCESSED_DIR}/full161_rare_tail_routing_table.csv")
genre_inventory_df = pd.read_csv(f"{PROCESSED_DIR}/full_genre_inventory.csv")
full_master_df = pd.read_csv(f"{PROCESSED_DIR}/multilabel_full_master_table.csv")

# Reference features metadata
features_reference = pd.read_csv(
    f"{RAW_METADATA_DIR}/features.csv",
    header=[0, 1, 2],
    index_col=0
)

print("Structured model loaded.")
print("Structured scaler loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Final frozen config loaded.")
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)
```

```
Structured model loaded.
Structured scaler loaded.
Audio model loaded.
Candidate labels: 150
Final frozen config loaded.
Rare-tail router shape: (13, 26)
Genre inventory shape: (163, 11)
Full master shape: (81574, 170)
Reference features shape: (106574, 518)
```

In [4]:

```
# ============================================================
# 4. PREPARE LOOKUPS
# ============================================================
```

```
genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)
genre_name_map = dict(zip(genre_inventory_df["genre_id"],
genre_inventory_df["genre_name"]))

candidate_label_ids = [int(col.replace("genre_", "")) for col in
candidate_label_cols]
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
enumerate(candidate_label_cols)}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

# Flatten reference feature columns
features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df.index = reference_feature_df.index.astype(int)
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)
reference_feature_means = reference_feature_df.mean(axis=0)
reference_feature_columns = list(reference_feature_df.columns)

full_master_indexed = full_master_df.set_index("track_id", drop=False)

fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

print("Fallback rare-tail labels:", len(fallback_rare_ids))
print("Inventory-only rare-tail labels:", inventory_only_df.shape[0])
print("Structured reference feature columns:", len(reference_feature_columns))
```

```
Fallback rare-tail labels: 10
Inventory-only rare-tail labels: 3
Structured reference feature columns: 518
```

In [5]:

```
# ============================================================
# 5. RESOLVE INPUT AUDIO
# ============================================================

if MODE == "existing_fma_track":
    if TRACK_ID_TO_TEST not in full_master_indexed.index:
```

```
        raise ValueError(f"track_id {TRACK_ID_TO_TEST} not found in full_master
table.")

    input_row = full_master_indexed.loc[TRACK_ID_TO_TEST]
    AUDIO_PATH = input_row["audio_path"]
    ACTIVE_TRACK_ID = int(input_row["track_id"])
    INPUT_SOURCE = "existing_fma_track"

elif MODE == "external_file":
    AUDIO_PATH = EXTERNAL_AUDIO_PATH
    ACTIVE_TRACK_ID = None
    INPUT_SOURCE = "external_file"

else:
    raise ValueError("MODE must be either 'existing_fma_track' or
'external_file'.")

print("Input source:", INPUT_SOURCE)
print("Audio path:", AUDIO_PATH)
print("Track ID:", ACTIVE_TRACK_ID)
```

```
Input source: external_file
Audio path: C:\Users\jdevo\Downloads\Bob Marley - Is This Love (Official Music
Video).mp3
Track ID: None
```

In [6]:

```
# ============================================================
# 6. HELPER FUNCTIONS
# ============================================================

def load_audio(file_path, sr=SR, duration=DURATION):
    y, sr_loaded = librosa.load(file_path, sr=sr, mono=True, duration=duration)
    if y is None or len(y) == 0:
        raise ValueError(f"Could not load usable audio from: {file_path}")
    return y, sr_loaded

def build_mel_input(y, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH,
max_frames=MAX_FRAMES):
    mel = librosa.feature.melspectrogram(
        y=y,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]
```

```
    return mel_db.astype(np.float32)[None, :, :, None]

def safe_stat_vector(arr_2d, stat_name):
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)
    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)
    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)
    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)
    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)
    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")
    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")
    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)
    out[~np.isfinite(out)] = 0.0
    return out.astype(np.float32)

def build_feature_matrices(y, sr=SR):
    y = np.asarray(y, dtype=np.float64)
    y_harmonic = librosa.effects.harmonic(y)

    mats = {}
    mats["chroma_stft"] = librosa.feature.chroma_stft(y=y, sr=sr)
    mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y, sr=sr)
    mats["chroma_cens"] = librosa.feature.chroma_cens(y=y, sr=sr)
    mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    mats["mfcc"] = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
    mats["rms"] = librosa.feature.rms(y=y)
    mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y, sr=sr)
    mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y, sr=sr)
    mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y, sr=sr)
    mats["zcr"] = librosa.feature.zero_crossing_rate(y)
    return mats

def build_structured_feature_vector(y, reference_columns, reference_means, sr=SR):
    feature_mats = build_feature_matrices(y, sr=sr)
    row_dict = {}

    for col in reference_columns:
        parts = col.split("_")
        component_idx = int(parts[-1]) - 1
        stat_name = parts[-2]
        feature_name = "_".join(parts[:-2])
```

```
        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])

    return X_one.astype(np.float32)

def scores_to_pseudoprobs(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def get_structured_scores(model, X_scaled):
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)
    elif hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)
    else:
        raise ValueError("Structured model supports neither predict_proba nor
decision_function.")
    return np.asarray(scores)

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

def decode_stage1(prob_matrix, threshold):
    return (prob_matrix >= threshold).astype(np.uint8)

def build_stage2_scores_baseline_prior(stage1_probs, router_df,
candidate_index_map, trigger_threshold):
    rows = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])
        anchor_idx = candidate_index_map[anchor_id]
        anchor_prob = float(stage1_probs[0, anchor_idx])

        if anchor_prob < trigger_threshold:
            continue

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])
```

```
        prior_strength = max(p_anchor, p_root)
        stage2_score = anchor_prob * prior_strength

        rows.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "prior_strength": prior_strength,
            "stage2_score": stage2_score
        })

    if len(rows) == 0:
        return pd.DataFrame(columns=[
            "rare_tail_genre_id", "rare_tail_genre_name",
            "anchor_candidate_id", "anchor_candidate_name",
            "anchor_prob", "prior_strength", "stage2_score"
        ])

    return pd.DataFrame(rows).sort_values(
        ["stage2_score", "anchor_prob"],
        ascending=False
    ).reset_index(drop=True)

def get_ground_truth_for_fma_track(track_id):
    row = full_master_indexed.loc[track_id]

    true_candidate_ids = []
    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    true_candidate_names = [genre_name_map.get(gid, str(gid)) for gid in
true_candidate_ids]

    true_rare_ids = []
    for col in fallback_rare_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    true_rare_names = [genre_name_map.get(gid, str(gid)) for gid in true_rare_ids]

    return {
        "track_id": int(track_id),
        "true_candidate_ids": true_candidate_ids,
        "true_candidate_names": true_candidate_names,
        "true_rare_tail_ids": true_rare_ids,
        "true_rare_tail_names": true_rare_names
    }
```

In [7]:

```
# ============================================================
# 7. LOAD AUDIO AND BUILD MODEL INPUTS
```

```
# ============================================================

y, sr_loaded = load_audio(AUDIO_PATH, sr=SR, duration=DURATION)

X_audio_input = build_mel_input(
    y,
    sr=sr_loaded,
    n_mels=N_MELS,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    max_frames=MAX_FRAMES
)

X_structured_one = build_structured_feature_vector(
    y,
    reference_feature_columns,
    reference_feature_means,
    sr=sr_loaded
)

X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

print("Loaded audio length (samples):", len(y))
print("Audio input shape:", X_audio_input.shape)
print("Structured feature vector shape:", X_structured_one.shape)
print("Structured scaled shape:", X_structured_scaled.shape)

display(X_structured_one.iloc[:, :20])
```

```
Loaded audio length (samples): 330750
Audio input shape: (1, 64, 324, 1)
Structured feature vector shape: (1, 518)
Structured scaled shape: (1, 518)
```

	chroma_cens_kurtosis_01	chroma_cens_kurtosis_02	chroma_cens_kurtosis_03	chroma_cens_ku
0	-0.703918	-0.349073	-1.009305	

In [8]:

```
# ============================================================
# 8. RUN STRUCTURED BRANCH
# ============================================================

structured_scores = get_structured_scores(structured_model, X_structured_scaled)
structured_probs = scores_to_pseudoprobs(structured_scores)

print("Structured score shape:", structured_scores.shape)
print("Structured probability shape:", structured_probs.shape)
```

```
Structured score shape: (1, 150)
Structured probability shape: (1, 150)
```

In [9]:

```
# ============================================================
# 9. RUN AUDIO BRANCH
# ============================================================

audio_probs = audio_model.predict(X_audio_input, verbose=0)

print("Audio probability shape:", audio_probs.shape)
```

```
Audio probability shape: (1, 150)
```

In [10]:

```
# ============================================================
# 10. RUN FINAL STAGE-1 HYBRID
# ============================================================

stage1_probs = fuse_probabilities(
    structured_probs,
    audio_probs,
    STAGE1_STRUCTURED_WEIGHT,
    STAGE1_AUDIO_WEIGHT
)

stage1_pred = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

print("Stage-1 fused probability shape:", stage1_probs.shape)
print("Stage-1 hard prediction shape:", stage1_pred.shape)
print("Number of predicted Stage-1 labels:", int(stage1_pred.sum()))
```

```
Stage-1 fused probability shape: (1, 150)
Stage-1 hard prediction shape: (1, 150)
Number of predicted Stage-1 labels: 4
```

In [11]:

```
# ============================================================
# 11. BUILD STAGE-1 RESULTS TABLES
# ============================================================

candidate_rows = []

for j, col in enumerate(candidate_label_cols):
    gid = int(col.replace("genre_", ""))
    candidate_rows.append({
        "genre_id": gid,
        "genre_name": genre_name_map.get(gid, str(gid)),
        "structured_probability": float(structured_probs[0, j]),
        "audio_probability": float(audio_probs[0, j]),
        "hybrid_probability": float(stage1_probs[0, j]),
        "predicted_stage1": int(stage1_pred[0, j])
    })
```

```
candidate_results_df = pd.DataFrame(candidate_rows).sort_values(
    ["predicted_stage1", "hybrid_probability"],
    ascending=[False, False]
).reset_index(drop=True)

predicted_candidate_df = candidate_results_df[
    candidate_results_df["predicted_stage1"] == 1
].copy().reset_index(drop=True)

top_candidate_df = candidate_results_df.head(TOP_N_CANDIDATES).copy()

print("Predicted Stage-1 candidate labels:")
display(predicted_candidate_df)

print(f"Top {TOP_N_CANDIDATES} candidate labels by hybrid probability:")
display(top_candidate_df)
```

```
Predicted Stage-1 candidate labels:
```

	genre_id	genre_name	structured_probability	audio_probability	hybrid_probability	predicte
0	12	Rock	1.256079e-02	0.457992	0.413449	
1	15	Electronic	4.000750e-05	0.311248	0.280128	
2	38	Experimental	5.646575e-03	0.230757	0.208246	
3	21	Hip-Hop	2.061154e-09	0.230338	0.207304	

```
Top 10 candidate labels by hybrid probability:
```

	genre_id	genre_name	structured_probability	audio_probability	hybrid_probability	predi
0	12	Rock	1.256079e-02	0.457992	0.413449	
1	15	Electronic	4.000750e-05	0.311248	0.280128	
2	38	Experimental	5.646575e-03	0.230757	0.208246	
3	21	Hip-Hop	2.061154e-09	0.230338	0.207304	
4	76	Experimental Pop	1.000000e+00	0.071711	0.164540	
5	10	Pop	7.601333e-03	0.181628	0.164225	
6	27	Lo-Fi	1.000000e+00	0.041244	0.137119	
7	25	Punk	2.061154e-09	0.148358	0.133522	
8	41	Electroacoustic	1.000000e+00	0.016965	0.115268	
9	107	Ambient	9.999997e-01	0.016218	0.114596	

In [12]:

```
# ============================================================
# 12. RUN FINAL STAGE-2 FALLBACK
# ============================================================

stage2_scores_df = build_stage2_scores_baseline_prior(
    stage1_probs,
    fallback_router_df,
    candidate_id_to_index,
    trigger_threshold=STAGE2_ANCHOR_TRIGGER_THRESHOLD
)

stage2_suggestions_df =
stage2_scores_df.head(STAGE2_TOP_K).copy().reset_index(drop=True)

print("All Stage-2 candidate rare-tail scores:")
display(stage2_scores_df)

print("Final Stage-2 rare-tail suggestions:")
display(stage2_suggestions_df)
```

All Stage-2 candidate rare-tail scores:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	ancho
0	176	Pacific	2	International	0.

Final Stage-2 rare-tail suggestions:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	ancho
0	176	Pacific	2	International	0.

In [13]:

```
# ============================================================
# 13. INVENTORY-ONLY LABELS
# ============================================================

inventory_only_labels_df = inventory_only_df[
    ["rare_tail_genre_id", "rare_tail_genre_name", "anchor_candidate_name",
"root_candidate_name", "fallback_mode"]
].copy()

print("Inventory-only rare-tail labels:")
display(inventory_only_labels_df)
```

Inventory-only rare-tail labels:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	fallb
0	174	South Indian Traditional	Indian	International	Inve
1	175	Bollywood	Indian	International	Inve
2	178	Be-Bop	Jazz	Jazz	Inve

In [14]:

```
# ============================================================
# 14. GROUND TRUTH CHECK FOR FMA TRACKS
# ============================================================

ground_truth = None

if INPUT_SOURCE == "existing_fma_track" and ACTIVE_TRACK_ID is not None:
    ground_truth = get_ground_truth_for_fma_track(ACTIVE_TRACK_ID)

    print("Ground truth:")
    print(json.dumps(ground_truth, indent=2))
else:
    print("No ground truth shown because MODE is external_file.")
```

```
No ground truth shown because MODE is external_file.
```

In [15]:

```
# ============================================================
# 15. BUILD FINAL PIPELINE SUMMARY
# ============================================================

stage1_candidate_ids = predicted_candidate_df["genre_id"].astype(int).tolist()
stage1_candidate_names = predicted_candidate_df["genre_name"].tolist()

stage2_rare_tail_ids =
stage2_suggestions_df["rare_tail_genre_id"].astype(int).tolist() if
len(stage2_suggestions_df) > 0 else []
stage2_rare_tail_names = stage2_suggestions_df["rare_tail_genre_name"].tolist() if
len(stage2_suggestions_df) > 0 else []
stage2_rare_tail_scores = stage2_suggestions_df["stage2_score"].round(6).tolist()
if len(stage2_suggestions_df) > 0 else []

final_summary = {
    "input_source": INPUT_SOURCE,
    "audio_path": AUDIO_PATH,
    "track_id": ACTIVE_TRACK_ID,
    "stage1_config": {
        "model_name": final_project_config["stage1_primary_system_name"],
        "structured_weight": STAGE1_STRUCTURED_WEIGHT,
        "audio_weight": STAGE1_AUDIO_WEIGHT,
        "threshold": STAGE1_THRESHOLD
    },
```

```
    "stage2_config": {
        "router_name": final_project_config["stage2_router_name"],
        "strategy": STAGE2_FINAL_STRATEGY,
        "anchor_trigger_threshold": STAGE2_ANCHOR_TRIGGER_THRESHOLD,
        "top_k": STAGE2_TOP_K
    },
    "stage1_candidate_label_count": len(stage1_candidate_ids),
    "stage1_candidate_label_ids": stage1_candidate_ids,
    "stage1_candidate_label_names": stage1_candidate_names,
    "stage2_rare_tail_suggestion_count": len(stage2_rare_tail_ids),
    "stage2_rare_tail_suggestion_ids": stage2_rare_tail_ids,
    "stage2_rare_tail_suggestion_names": stage2_rare_tail_names,
    "stage2_rare_tail_suggestion_scores": stage2_rare_tail_scores,
    "inventory_only_rare_tail_labels":
inventory_only_labels_df["rare_tail_genre_name"].tolist()
}

if ground_truth is not None:
    final_summary["ground_truth"] = ground_truth

print("Final pipeline summary:")
print(json.dumps(final_summary, indent=2))
```

```
Final pipeline summary:
{
  "input_source": "external_file",
  "audio_path": "C:\\Users\\jdevo\\Downloads\\Bob Marley - Is This Love (Official
Music Video).mp3",
  "track_id": null,
  "stage1_config": {
    "model_name": "Expanded Hybrid Global Threshold",
    "structured_weight": 0.1,
    "audio_weight": 0.9,
    "threshold": 0.2
  },
  "stage2_config": {
    "router_name": "Rare-tail fallback router",
    "strategy": "baseline_prior",
    "anchor_trigger_threshold": 0.05,
    "top_k": 1
  },
  "stage1_candidate_label_count": 4,
  "stage1_candidate_label_ids": [
    12,
    15,
    38,
    21
  ],
  "stage1_candidate_label_names": [
    "Rock",
    "Electronic",
    "Experimental",
    "Hip-Hop"
  ],
  "stage2_rare_tail_suggestion_count": 1,
  "stage2_rare_tail_suggestion_ids": [
    176
  ],
  "stage2_rare_tail_suggestion_names": [
    "Pacific"
  ],
  "stage2_rare_tail_suggestion_scores": [
    0.000456
  ],
  "inventory_only_rare_tail_labels": [
    "South Indian Traditional",
    "Bollywood",
    "Be-Bop"
  ]
}
```

In [16]:

```
# ============================================================
# 16. SAVE FINAL OUTPUTS
# ============================================================

os.makedirs(PROCESSED_DIR, exist_ok=True)

candidate_results_df.to_csv(
```

```
    f"{PROCESSED_DIR}/final_pipeline_candidate_results.csv",
    index=False
)

predicted_candidate_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage1_predictions.csv",
    index=False
)

stage2_scores_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage2_all_scores.csv",
    index=False
)

stage2_suggestions_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage2_suggestions.csv",
    index=False
)

inventory_only_labels_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_inventory_only_labels.csv",
    index=False
)

summary_df = pd.DataFrame([{
    "input_source": final_summary["input_source"],
    "audio_path": final_summary["audio_path"],
    "track_id": final_summary["track_id"],
    "stage1_candidate_label_count": final_summary["stage1_candidate_label_count"],
    "stage2_rare_tail_suggestion_count":
final_summary["stage2_rare_tail_suggestion_count"]
}])

summary_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_summary.csv",
    index=False
)

with open(f"{PROCESSED_DIR}/final_pipeline_summary.json", "w") as f:
    json.dump(final_summary, f, indent=2)

print("Saved final pipeline inference outputs.")
```

```
Saved final pipeline inference outputs.
```